

Website Audit & Lead-Scoring Pipeline

Complete Technical Reference

Covers **website_audit.py** (single-site auditor engine) and **batch_website_audit.py** (batch runner for a leads CSV) — every check, every score, every CLI flag, and every output column, explained end to end.

Project: Sanjay Digital Works — AI Automation Client-Acquisition Pipeline

Files covered: `website_audit.py`, `batch_website_audit.py`

Purpose: Turn a list of scraped local-business leads into a scored, evidence-backed audit of each business's website — health issues, and automation/conversion gaps — ready to drive outreach.

Contents

1. System Overview — how the two scripts fit together
2. `website_audit.py` — the single-site audit engine
 - 2.1 Fetching a page reliably (static → retry → headless → give up honestly)
 - 2.2 Lighthouse (performance & accessibility)
 - 2.3 Every individual check, what it looks for, why it matters
 - 2.4 Scoring — Website Health vs. Automation & Conversion Opportunity
 - 2.5 Reports produced (console, JSON, HTML, outreach draft)
3. `batch_website_audit.py` — running it across a whole leads CSV
 - 3.1 How it reuses `website_audit.py` without duplicating logic
 - 3.2 Fast mode vs. full Lighthouse mode
 - 3.3 CLI flags reference
 - 3.4 Output CSV column reference
 - 3.5 Crash-safety: incremental writes and `--resume`
 - 3.6 Interactive mode (no arguments / double-click)
4. How to run it — command cheat-sheet
5. Known limitations & things worth a manual check
6. Troubleshooting log (issues actually hit, and how they were solved)

1. System Overview

The pipeline is two scripts that work together. **website_audit.py** is the engine: point it at one URL and it produces a complete audit of that single website. **batch_website_audit.py** is a thin wrapper around that engine: point it at a leads CSV (the kind your scraper already produces, with a *Website* column) and it runs the engine once per lead, collecting the important results into one new CSV — one row per business.

Nothing about the underlying checks is duplicated between the two files. The batch script literally imports the classes from `website_audit.py` and re-uses them, so any future improvement to the single-site engine automatically applies to the batch run too.

Data flow

Step	What happens	File responsible
1	Leads CSV is read (Business Name, Website, Address, Phone, ...)	<code>batch_website_audit.py</code>
2	For each lead with a website, normalize the URL and build an <code>AuditConfig</code>	<code>batch_website_audit.py</code>
3	<code>WebsiteAuditor(config).run()</code> fetches the page, runs every check, scores it	<code>website_audit.py</code>
4	The key fields of the result are flattened into one output row	<code>batch_website_audit.py</code>
5	That row is written to the output CSV immediately (not buffered)	<code>batch_website_audit.py</code>
6	Repeat for the next lead, with a small polite delay in between	<code>batch_website_audit.py</code>

In other words: **website_audit.py** answers "what's wrong with this one website?", and **batch_website_audit.py** answers "run that question against every lead I scraped, and hand me back a spreadsheet."

2. website_audit.py — the single-site audit engine

This file requires no API keys. It's built entirely on **requests** + **BeautifulSoup** for the HTML-level checks, with an optional **Playwright** headless-browser fallback for JavaScript-heavy sites, and an optional **Lighthouse-via-Docker** pass for real performance/accessibility scores.

2.1 Fetching a page reliably

Real-world small-business websites are messy: some are plain HTML, some are single-page React/Wix apps that render with JavaScript, some sit behind Cloudflare's bot-challenge page, and some just redirect. A naive `requests.get()` would silently "succeed" on a blocked or empty page and then every check below it would report false negatives ("no title", "no forms", "no links") — which would poison the audit without any error ever being raised. The fetch logic exists specifically to avoid that trap.

The fetch process, in order:

- **1. Plain GET request** with a normal desktop User-Agent.
- **2. Block-page detection** — the raw HTML/title is scanned against a list of known phrases ("checking your browser", "access denied", "just a moment", CAPTCHA/Turnstile/hCaptcha markers, etc.). If matched, the page is flagged as blocked rather than trusted as-is.
- **3. Retry with fuller browser-style headers** (Accept, Sec-Fetch-*, etc.) — some naive bot-blockers only check for these headers being present at all.
- **4. Headless-browser retry (Playwright/Chromium)**, if installed — a real browser engine can pass some JS-based challenges that a bare HTTP client never can. This is also used whenever the raw HTML looks like an empty "JS shell" (very little visible text, very few real tags) or contains a JavaScript redirect.
- **5. Meta-refresh / same-host redirect following** — a `<meta http-equiv="refresh">` tag pointing at the same domain is followed automatically.
- **6. Honest failure** — if none of the above gets past a challenge, the tool does *not* silently pretend it worked. It records a `fetch_warning` explaining exactly what happened (e.g. "still looks blocked after a headless retry — this likely needs a human-facing CAPTCHA, verify manually") so you know to double-check that lead by hand instead of trusting a false "0 issues found".

This is exactly the mechanism you saw fire in your real runs — for example *Iron Rooster Muay Thai* (Instagram page, JS shell → headless render succeeded) and *Ninja Clint Boxing* (Cloudflare Turnstile → headless retry succeeded), versus *Modern Martial Arts – Upper East Side*, where even the headless retry timed out and it had to fall back to the (incomplete) static HTML, which is exactly why that one carries a `fetch_warning` in the output.

2.2 Lighthouse (performance & accessibility)

If Docker is installed and running, the script shells out to the `femtapixel/google-lighthouse` Docker image to get real Google Lighthouse scores — the same engine behind Google PageSpeed Insights.

- Runs N times per form factor (default 3) because Lighthouse scores are noisy run-to-run; the **median** score across runs is used, and the single run closest to that median is kept as the "representative" report (for its screenshot and detailed metrics).
- Runs for **mobile** always; **desktop** is optional (`check_desktop`).
- Each run is wrapped in a timeout (default 180s) and its own try/except, so one slow or failing Lighthouse run never crashes the whole audit — it's recorded as a `lighthouse_error` and the audit continues with whatever runs did succeed.

- If Docker isn't running at all, Lighthouse is skipped entirely and those two categories are marked **untested** (excluded from the health-score denominator) rather than being scored as a 0/100 failure — an untested check is treated very differently from a failed one.

This is also the slow part. Each Lighthouse run spins up a fresh Docker container and takes anywhere from ~15 seconds to 90+ seconds depending on the site and your machine. At 3 runs × 2 form factors, that's up to ~6 minutes per site — which is exactly what made the first 30-lead batch run take over an hour. See Section 3.2 for how batch mode handles this.

2.3 Every individual check

Below is every check the engine performs on the fetched HTML, grouped by area. Each one either contributes to a score (Section 2.4) or generates a structured **Issue** (category, problem, evidence, why it matters, recommended fix, priority) that flows straight into the reports.

Check	What it looks at	Flags an issue when...
SSL certificate	Live TLS handshake to the domain on port 443	Certificate missing/invalid, or expiring within 21 days
Security headers	HSTS, CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy	Any of the 6 recommended headers is missing
Mixed content	http:// resource references on an https:// page	Any insecure resource reference found
HTTP→HTTPS redirect	Requesting the http:// version of the same URL	It doesn't land on an https:// URL
robots.txt / sitemap.xml	GET /robots.txt and /sitemap.xml (or the sitemap URL declared inside robots.txt)	Either file is missing (robots.txt = LOW priority, sitemap = MEDIUM)
Favicon	<link rel="icon"> tag, else GET /favicon.ico	Neither is found

SEO / technical / mobile

Check	What it looks at	Flags an issue when...
Title tag	<title> content and length	Missing, or outside ~10-60 characters
Meta description	<meta name="description">	Missing, or outside ~50-160 characters
Canonical tag	<link rel="canonical">	Missing
H1 usage	Count of <h1> tags	Not exactly one H1 (zero or several)
Alt text	% of tags missing an alt attribute	20%+ of images have no alt text
noindex	<meta name="robots"> content	Contains noindex (flagged CRITICAL — may be unintentional)
Open Graph tags	og:title, og:description, og:image	All three are missing (social share previews will look blank)

Check	What it looks at	Flags an issue when...
Mobile viewport	<meta name="viewport">	Missing (CRITICAL — desktop-only rendering on phones)

Automation & conversion signals

Check	What it looks at	Flags an issue when...
AI chatbot / live chat	Known script/CDN signatures for 13 providers (Intercom, Drift, Tidio, Zendesk, LiveChat, Crisp, Tawk.to, HubSpot, Freshchat, Olark, ManyChat, Chatbot.com, Facebook Messenger)	None of the signatures are found (HIGH priority)
WhatsApp CTA	wa.me/ or api.whatsapp.com/send links in the HTML	Neither pattern found (MEDIUM)
Appointment booking	Known booking-widget domains (Calendly, Acuity, Setmore, Booksy, Fresha, Vagaro, etc.) or "book an appointment"-style phrasing	None found (MEDIUM)
Lead-capture form	<form> elements that contain real input/textarea fields (not just a submit button)	No meaningful form found (HIGH); a form with 8+ fields is separately flagged as too long (LOW)
Phone CTA	 links	None found (MEDIUM) — on mobile, a non-clickable number adds friction
Trust signals	"testimonial/review/5-star/trusted by" keywords in visible text, or AggregateRating schema	Neither found (MEDIUM)
Local Business schema	JSON-LD <script type="application/ld+json"> blocks with a LocalBusiness-family @type	No such schema found (MEDIUM); found-but-missing opening hours is a separate LOW issue

Internal link health

Check	What it looks at	Flags an issue when...
Internal link count	Number of same-domain links found in the HTML (capped at max_links, default 25)	Fewer than 8 found on a page over 3000 bytes — usually means JS-rendered navigation wasn't visible to this tool (LOW, and a hint the results below may be incomplete)
Broken internal links	HTTP status of every same-domain link found (with a small delay between requests, and a retry on 429 rate-limits)	Status code 400+ (HIGH if 3+, else MEDIUM). Cloudflare's obfuscated email-protection links are explicitly excluded — they legitimately 404 for any non-browser client and are not a real broken link.

Check	What it looks at	Flags an issue when...
403-Forbidden links	Same crawl as above	Reported <i>separately</i> from broken links, as LOW / "needs manual verification" — a 403 usually means the automated request itself got blocked, not that the page is actually down for a real visitor

Note: **batch_website_audit.py** always skips this internal-link crawl (Section 3.1) — it's the slowest per-site check and the least useful signal for a first-pass outreach list.

Business-info extraction

Alongside the checks above, the engine tries to pull the business's own name, phone, and address — first from JSON-LD schema, then Open Graph og:site_name, then falling back to a cleaned-up version of the page <title>. This is used to personalize the outreach message draft (Section 2.5).

What is deliberately NOT auto-checked

The engine is explicit about the difference between "checked and passed" and "couldn't be checked automatically." These are always listed in every report as **manual_review_needed**, because they require actual human or visual judgement:

- First impression / visual hierarchy
- Branding and image quality
- CTA placement and clarity (above/below the fold)
- Mobile layout rendering (overflow, spacing, sticky elements)
- Navigation/menu usability as a real visitor would experience it
- Competitor / modern-design comparison
- Screenshots of concrete problems

2.4 Scoring

Two separate scores are produced — deliberately never blended into one number — so a technically excellent site is never punished for lacking chatbots, and a site full of automation isn't given credit for basic technical hygiene it doesn't have.

Website Health Score (out of 100)

Category	Weight	Based on
Performance	20	Lighthouse performance score (mobile weighted 60%, desktop 40%, if run)
SEO (technical)	20	50% Lighthouse SEO score + 50% manual checks (title, description, canonical, H1, alt text, noindex)
Accessibility	15	Lighthouse accessibility score
UX / technical	15	Viewport tag present + at least one H1
Security	10	SSL valid, all 6 security headers, no mixed content, HTTP→HTTPS redirect works
Technical health	10	robots.txt found, sitemap found, favicon found, % of internal links not broken
Local schema	10	Full 10 points if any LocalBusiness-type schema is found, else 0

Untested categories are excluded from the total, not scored as zero. If Lighthouse didn't run, Performance (20) and Accessibility (15) are subtracted from the maximum, so a site's health score is always out of however many categories were actually testable for it (e.g. 65/100 in fast mode with Lighthouse off, as seen in your batch runs).

Automation & Conversion Opportunity Score (out of 100)

Category	Weight	Based on
AI chatbot / live chat	25	Full points if any chatbot signature detected, else 0
Lead-capture form	15	Full points if a real form is found
WhatsApp CTA	15	Full points if a wa.me/ or api.whatsapp.com link is found
Appointment booking	15	Full points if a booking widget/phrasing is found
Phone CTA	10	Full points if a tel: link is found
Trust signals	10	Full points if testimonial/review content or rating schema is found
Local schema completeness	10	Scaled 0-10 by how many of {schema found, phone found, address found, hours found} are present

A **low** opportunity score means the business is missing most of these systems — which is exactly the pain point an automation-services pitch is built around. A **high** health score alongside a **low** opportunity score (e.g. Bua Boxing: 45.6 health / 2.5 opportunity in your real run) is often the ideal outreach target: a site that's technically fine, run by someone competent, but simply hasn't automated anything yet.

2.5 Reports produced

- **Console output** — human-readable summary: both scores, full breakdown by category, every issue sorted by priority (CRITICAL → HIGH → MEDIUM → LOW), the manual-review checklist, Lighthouse metrics, SSL info, security headers, and internal-link stats.
- **full_report.json** — the entire AuditResult serialized, for programmatic use.
- **audit_report.html** — a styled, client-presentable HTML report with both scores as big numbers, category breakdown tables, and color-coded issue cards (red=CRITICAL, orange=HIGH, yellow=MEDIUM, green=LOW).
- **outreach_message.txt** — an auto-drafted cold-outreach message, personalized with the extracted business name and its top 3 issues.

3. batch_website_audit.py — running it across a whole leads CSV

This script takes the single-site engine above and runs it once per row of a leads CSV (must have a *Website* column — the same format your `gmaps_scrape_run` pipeline already produces), writing one summarized row per lead to a new CSV.

3.1 How it reuses `website_audit.py` without duplicating logic

The script inserts its own folder onto `sys.path` and does a direct Python import:

```
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from website_audit import AuditConfig, WebsiteAuditor, normalize_url, PRIORITY_ORDER
```

This is why the two files must sit in the same folder — Python needs to find `website_audit.py` right next to `batch_website_audit.py`. Every check, every score, and every fetch-retry behavior described in Section 2 runs completely unchanged; nothing is copy-pasted or re-implemented.

The one exception: the internal broken-link crawl (Section 2.3) is disabled for every batch run. This is done by **monkeypatching** a single method on the imported class before any audits run:

```
def _skip_link_crawl(self, soup, raw_html):
    return {"checked_count": 0, "broken": [], "broken_count": 0, ...}

WebsiteAuditor._crawl_internal_links = _skip_link_crawl
```

This replaces just that one method at runtime, in memory, for this process only — it does not edit `website_audit.py` on disk, and running `website_audit.py` directly (single-site mode) is completely unaffected.

3.2 Fast mode vs. full Lighthouse mode

This is the setting that matters most for how long a batch run takes. Lighthouse is **off by default** in the batch script.

Mode	How to get it	Per-site time (observed)	What you lose
Fast (default)	no flag needed	1-15 seconds	No Lighthouse performance/accessibility scores; health score is out of 65, not 100
Full Lighthouse	<code>--full-lighthouse</code>	~100-280 seconds	Nothing — full 100-point health score, real PageSpeed-style numbers

In practice: use fast mode to triage all 30+ leads in a couple of minutes, then re-run `--full-lighthouse` on just your shortlist of the best 3-5 outreach targets if you want real performance numbers for the pitch.

3.3 CLI flags reference

Flag	Default	What it does
<code>input_csv</code> (positional)	prompts if omitted	Path to the leads CSV

Flag	Default	What it does
--out PATH	<input>_audited.csv	Output CSV path
--full-lighthouse	off	Also run Lighthouse (slow, see 3.2)
--runs N	1	Lighthouse runs per form factor (only used with --full-lighthouse)
--no-desktop	off	With --full-lighthouse, skip the desktop pass (mobile only)
--limit N	all leads	Only audit the first N leads — for testing before a full run
--resume	off	Skip websites already present in the output CSV (Section 3.5)
--delay SECONDS	0.3	Pause between sites, to be polite to the sites/your network
--reports-dir PATH	./batch_reports	Where Lighthouse JSON/screenshots are written, if enabled
--lighthouse-timeout SECONDS	180	Per-Lighthouse-run timeout before it's aborted

3.4 Output CSV column reference

Column	Meaning
business_name	From the leads CSV's Business Name column
website	The URL that was audited, exactly as it appeared in the input
audit_status	OK / FAILED / SKIPPED_NO_WEBSITE
error	Exception message, only populated if audit_status = FAILED
health_score / max_health_score	Website Health Score and its denominator (see 2.4)
opportunity_score / max_opportunity_score	Automation & Conversion Opportunity Score, always out of 100
ssl_valid	True/False — is the SSL certificate currently valid
has_chatbot / chatbot_providers	Whether a chat widget was detected, and which provider(s) by name
has_whatsapp	Whether a wa.me / api.whatsapp.com link was found
has_lead_form	Whether a real contact/quote form was found
has_phone_cta	Whether a clickable tel: link was found
has_appointment_booking	Whether a booking widget or booking phrasing was found
has_trust_signals	Whether testimonials/reviews or rating schema were found
has_local_schema	Whether LocalBusiness-family JSON-LD schema was found

Column	Meaning
critical_issue_count / high_issue_count	How many CRITICAL / HIGH priority issues were raised
top_issues	Up to 5 issues, worst-priority-first, as "PRIORITY: problem" separated by ";"
fetch_warning	Non-empty if the page may not have been fully/reliably fetched (Section 2.1) — treat that row's results with extra caution

3.5 Crash-safety: incremental writes and --resume

An early version of this script buffered all results in memory and wrote the output CSV only once, at the very end. That meant a hang or crash partway through the batch (which did happen once, mid-run) lost every result computed up to that point.

The fixed version now:

- Opens the output CSV once, up front, and calls `writer.writerow(...)` followed by `out_f.flush()` after **every single site** — so the file on disk is always current, even if the process is killed a moment later.
- Supports `--resume`: on start, it reads whatever rows already exist in the output CSV, builds a set of already-audited website URLs, and skips those leads entirely on this run — appending only the new results instead of starting over.

Practical recovery flow if a run stops (hang, Ctrl+C, closed window, Docker crash, etc.):

```
python batch_website_audit.py "gosom_run\qualified_leads_row1.csv" --resume
```

Same input CSV, same output path, just add `--resume` — it will only redo the leads that never got a row written.

3.6 Interactive mode (no arguments / double-click)

If the script is launched with *no command-line arguments at all* (e.g. double-clicked, or run via "Open with → Python"), it detects this (`len(sys.argv) == 1`) and:

- Prompts interactively: *"Enter the path to the leads CSV..."* — quotes around a pasted path are stripped automatically.
- Validates the path actually exists before continuing.
- At the very end, pauses with *"Press Enter to close..."* so the console window doesn't vanish before you can read the results — this only happens in no-argument mode; normal terminal runs exit immediately as usual.

4. How to run it — command cheat-sheet

Goal	Command
Single-site audit (console + JSON + HTML reports)	<code>python website_audit.py https://example.com</code>
Single-site, skip Lighthouse (fast)	<code>python website_audit.py https://example.com --skip-lighthouse</code>
Batch, fast mode, prompts for CSV path	<code>python batch_website_audit.py</code>
Batch, fast mode, path given directly	<code>python batch_website_audit.py "gosom_run\qualified_leads_row1.csv"</code>
Batch, test on first 2 leads only	<code>python batch_website_audit.py leads.csv --limit 2</code>
Batch, full Lighthouse (slow, best accuracy)	<code>python batch_website_audit.py leads.csv --full-lighthouse</code>
Resume a batch that stopped partway	<code>python batch_website_audit.py leads.csv --resume</code>
Batch, custom output filename	<code>python batch_website_audit.py leads.csv --out my_results.csv</code>

Requirements checklist before running a batch:

- `batch_website_audit.py` and `website_audit.py` must be in the **same folder**.
- For `--full-lighthouse`: Docker Desktop must be running (Lighthouse quietly marks itself "untested" instead of crashing if it isn't, but you won't get real scores).
- The input CSV needs a column literally named *Website* (case-insensitive) — it's matched automatically regardless of what other columns exist.
- First run may trigger a Windows Firewall prompt for `python.exe` — click "Allow access"; this is expected, since the script makes outbound web requests.

5. Known limitations & things worth a manual check

- **WhatsApp detection is pattern-based** (wa.me/ or api.whatsapp.com/send in the HTML). In one real 30-lead run, 0 out of 30 sites showed a WhatsApp CTA — plausible for this market, but also exactly the pattern you'd see if a WhatsApp icon linked through a redirect/shortener the pattern doesn't catch. Worth eyeballing a few sites by hand before using "no WhatsApp" as a firm outreach claim.
- **403-Forbidden internal links are not counted as broken**, by design — they're reported separately as "needs manual verification," since a 403 often just means the automated request itself got blocked (bot protection), not that the page is actually down for a real visitor.
- **Fast-mode health scores are out of 65, not 100** — Performance (20) and Accessibility (15) are excluded from both the score and its denominator when Lighthouse doesn't run, so a 45/65 in fast mode is not directly comparable to a 45/100 in full-Lighthouse mode. Compare scores earned under the same mode.
- **Manual-review items are never auto-scored** (Section 2.3) — first impression, visual hierarchy, CTA placement, and competitor comparison always require a human to actually look at the site.
- **Playwright must be installed separately** (pip install playwright && playwright install chromium) for the JS-rendering fallback to work at all. Without it, JS-heavy sites (React/Vue/Wix apps) may show false negatives on title/forms/links, flagged via a fetch_warning.
- **Business-name extraction can be imperfect** — it falls back to a cleaned page <title>, which occasionally captures a tagline instead of the actual business name (seen with El Dojo NYC in a real run, where the extracted name was "Martial Arts training Schedule...").

6. Troubleshooting log — issues actually hit, and how they were solved

Symptom	Cause	Fix
ModuleNotFoundError: No module named 'website_audit'	batch_website_audit.py was placed in a different folder than website_audit.py	Move both files into the same folder (this project uses Main_project/, next to website_audit.py)
PowerShell: "Cannot find path ...\\gosom_run\\gosom_run"	Ran cd gosom_run while the terminal prompt was already inside gosom_run	Check the current prompt path before cd-ing; use cd .. to go up a level instead
A Windows Firewall prompt appeared mid-run	First time python.exe makes an outbound network request, Windows flags it	Click "Allow access" (safe — it's the script's own web requests); the run resumes automatically afterward
The run went silent for several minutes mid-site	A Lighthouse Docker run hung or Docker itself became unresponsive	Confirm Docker Desktop is still running; Ctrl+C if truly frozen, then re-run in fast mode or with --resume once the incremental-write fix was in place
Full 30-lead batch was taking 1.5-2+ hours	--full-lighthouse (3 mobile + 3 desktop Docker runs per site) is inherently slow — this is expected behavior, not a bug	Switched to fast mode (Lighthouse off by default) — same 30 leads then completed in ~3-4 minutes
"RemoteDisconnected... /sitemap.xml" warning in the log	That specific site's server dropped the connection when checking for a sitemap	Harmless — the sitemap check fails gracefully for that one site and the rest of the audit continues normally

End of document. Both scripts are otherwise self-documented — the top docstring in each .py file mirrors the summaries above and stays in sync with the code.